PRD — Elara Commerce: Agentic Virtual Try-On Widget for Any E-Commerce Store

Nov 1, 2026 · @Saksham

1. Executive summary

We are building a plug-and-play AI stylist widget that any fashion e-commerce store can drop onto its site, submitted to the **eCommerce VTO** track of the YouCam API Skin AI & eCommerce VTO Hackathon (deadline 2 Nov 2026, 10:15 PM IST). A shopper opens the widget, uploads one full-body photo, answers two or three questions, and then talks to an agent in plain language ("beach trip outfit for my brother, under ₹4,000"). The agent assembles a complete outfit from that store's catalogue, renders it on the shopper's body with the YouCam Apparel Virtual Try-On API, and lets them swap any single piece, see a compatibility score, invite friends into a Style Crew to vote, and add the whole look to cart.

What makes this more than a try-on wrapper: the try-on is the *output* of an agentic shopping flow, not the feature. Three things judges have not seen combined before:

1. **Store-agnostic agent** – one embed script plus a product feed turns any catalogue (demo: ~500 Zara/H&M/Myntra SKUs) into a conversational stylist with try-on.
2. **Item-level swap with compatibility scoring** – change only the trousers on an already rendered look; the agent re-ranks alternatives and explains why they work.
3. **Style Crew** – shared try-on links where friends join, upload their own photo, discuss each piece in its own thread, and get a combined group photo; the social layer that turns "should I buy this?" into a purchase.

Underneath, every conversation, try-on and purchase feeds a persistent taste graph (Style DNA) that follows the shopper across stores, so recommendations improve from the first visit onward. The existing Elara product (app.joinelara.com) already proves the taste-graph, wardrobe and Style DNA ideas in a consumer app; this hackathon build moves that intelligence into the moment of purchase on a merchant's own site.

Demo focus: virtual try-on and agentic commerce. Skin AI is deliberately out of scope (narrower audience, lower store footfall).

2. Hackathon context and judging map

The brief asks for a functional web or mobile prototype that integrates at least one YouCam API and delivers clear consumer or retail value, explicitly not a surface-level wrapper, with Agentic AI workflows called out as a stretch. Our build answers every clause of that sentence.

Key facts from the Devpost page

Item	Detail
Hackathon	YouCam API Skin AI & eCommerce VTO Hackathon (Perfect Corp)
Track we enter	eCommerce VTO ("bring Virtual Try-On together into one seamless, intuitive experience")
Deadline	2 Nov 2026, 10:15 PM IST (8:45 AM PT)
Prizes	1st $2,500 · 2nd $1,000 · 3rd $500 · Women in Tech $1,000 · Rising Star (student only) $1,000
Participants	1,569 registered
Free credits	1,000 API units per registered participant (redeem code via email)
Must submit	Repo URL (public or shared with contact_event@PerfectCorp.com), text description, screenshots, 1–3 min demo video on YouTube, API used explained in video
Video rules	Must show the project running on the target device; no third-party trademarks, copyrighted music or material without permission
If we win	Exit interview + inclusion in a blog article

Two-submission strategy (from the planning call). Same core engine, two accounts, two visually distinct UIs: one in the general track, one for the Rising Star student award. The student award has far fewer eligible entrants. Each submission needs its own repo, video and description; see Section 23 for the decisions this forces.

Trademark warning for the video. The demo catalogue uses Zara/H&M/Myntra product imagery. Brand logos, names and product photos are third-party material. Either (a) strip brand names and use a fictional store name in the demo ("Northline", "TALA-style demo

store") or (b) use the existing TALA integration only if TALA has agreed. Recommended: fictional store name, real product photos with brand text cropped, royalty-free music.

How each judging criterion maps to our build

Criterion	What judges ask	Our answer in the demo
Technological implementation	Thorough, skilful API integration; non-trivial working code	Apparel VTO called in three distinct modes (single item on the user, full-outfit layered render, swap re-render on an existing result); async task polling, caching, credit budgeting; agent orchestrates API calls from natural language
Design	Complete, coherent product, not a proof of concept	Full flow from store page to cart: widget → onboarding → chat/studio → try-on → swap → crew → add to cart. Consistent design system, empty/loading/error states
Potential impact	Credible, specific problem for a real audience	Return rates and "does it suit me?" hesitation on fashion e-commerce; merchant-side: one script, any catalogue. Demo on a realistic 500-SKU store
Quality of idea	Creative, non-obvious API use; understanding of the problem space	Try-on as the output of an agent, item-level swap with compatibility scoring, Style Crew social proof, cross-store taste graph; team already operates Elara in this space

3. Problem statement

Fashion e-commerce still asks the shopper to do the stylist's job alone: find the item, imagine it on their own body, guess what goes with it, and decide without anyone to ask. Each gap costs the store a sale or a return.

Shopper pain

- *"Will this look good on me?"* Model photos show a 5′10″ professional, not the shopper. Static try-on tools exist but only answer this for one item at a time, after the shopper has already found it.
- *"What do I wear it with?"* Product pages sell pieces, not outfits. The shopper assembles a look by opening six tabs, and most abandon.
- *"I'm buying for someone else."* Gifting and family shopping (a brother, a partner, a child) break every recommendation engine built on the logged-in user's own history.
- *"Let me ask my friend."* The real decision happens in a WhatsApp group with screenshots. The store is absent from that conversation and loses it.
- *Cold start.* A first-time visitor has no history, so every "recommended for you" carousel is generic. The store has one visit to prove it understands them.

Store pain

- Return rates on apparel are high and fit/look expectations are a leading cause; every return erases margin.
- Existing VTO integrations are per-product buttons bolted onto a PDP. They do not change what the shopper finds, only how one item is previewed.
- Personalisation stacks need months of data engineering per store; small and mid-size brands never get there.
- Shopper intent ("beach trip, budget ₹4k, for my brother") is expressed in words that a filter sidebar cannot capture.

Why existing VTO is not enough

Virtual try-on today is a destination feature: the shopper must already know what they want, navigate to it, and press a button. The question shoppers actually have is upstream of that: *what should I even be trying on?* Answering it needs an agent that understands the occasion, the person, the budget and the catalogue, and then uses try-on as the way to show its answer. That is the gap this product fills, and the reason the try-on API is used as the output layer of an agent rather than as a standalone widget.

4. Product vision, positioning and name

Vision. Every fashion store gets a personal stylist that knows the shopper, shows every suggestion on the shopper's own body, and brings their friends into the decision, installed with one script tag.

One-line pitch for the submission. *An agentic AI stylist widget for any e-commerce store: describe the occasion, see complete outfits rendered on you with YouCam Virtual Try-On, swap any piece, shop it with your crew.*

Positioning. We sit between three existing categories and borrow the best of each:

Category	What it does	What it lacks	What we take from it
PDP try-on buttons (most VTO integrations)	Preview one item on a photo	Discovery, outfits, social	The YouCam render quality
Conversational shopping assistants	Natural-language search over a catalogue	Visual proof; the shopper still imagines the result	The agent loop and clarification
Consumer styling apps (Elara, Whering, Acloset)	Taste profile, wardrobe, daily picks	No presence at the point of purchase	Style DNA, wardrobe, taste graph

Name. Working title in this PRD: **Elara Commerce** (the B2B layer of Elara). Alternatives if a fresh brand is wanted for the general-track submission: *Fitly*, *Looksmith*, *TryCrew*. The student-track submission needs a different name and visual identity in any case (see Section 23). Demo store name: a fictional brand so no third-party trademark appears in the video.

Design principles

1. Show, don't list: every recommendation is a picture of the shopper wearing it, not a grid of product tiles.
2. Minimum questions: two or three before the first result; everything else is learned from choices.
3. One piece at a time: the shopper can change any single item without losing the rest of the look.
4. Store-agnostic: nothing in the widget knows about one catalogue; it reads whatever feed it is given.
5. The shopper owns their taste: Style DNA travels with them, not with the store.

5. Target users and personas

Three personas drive the demo; the first two appear on camera, the third is the buyer the judges should picture.

Persona	Who	Trigger	What they need from us	Demo moment
Priya, 24, first-time visitor	Shops a D2C fashion store for the first time, no account, no history	Has a Goa trip in 10 days and ₹4,000	Fast onboarding, outfits that fit her vibe, proof it looks good on *her*, a way to ask friends	Opens widget → Style DNA → chat → try-on → shares to crew
Arjun, 29, gifting shopper	Buying for his brother's birthday on the same store	Doesn't know sizes or style words, knows the occasion ("college farewell")	Agent infers the recipient's gender and context, asks only what it must, shows the recipient-appropriate outfit	Chat: "farewell outfit for my brother" → agent returns men's look, no gender question asked
Meera, store owner / e-com manager	Runs a 500-SKU Shopify/custom store	Return rate and low AOV; cannot build personalisation in-house	One script tag + product feed; no catalogue re-tagging; analytics on what shoppers try on and buy	Shown via the "plug and play" slide: embed snippet + feed JSON

Secondary persona: the Crew member. Priya's friend receives a link, opens the shared try-on without installing anything, uploads her own photo, votes and comments per item, and appears in the group photo. She converts from spectator to shopper inside the same session.

Explicitly not targeted in the demo: beauty/skincare shoppers (Skin AI track), B2B wholesale, kids' wear (child photos are excluded from try-on for safety and policy reasons; the agent will decline to render on a minor's photo).

6. Goals, non-goals and demo scope

Goals for the submission (measurable by the judges)

1. A judge can open the hosted demo store, complete onboarding, type one request and see a complete outfit rendered on the uploaded photo in under 60 seconds.
2. Every outfit piece can be swapped individually and the look re-renders without starting over.
3. A second person can join a Style Crew from a link, upload their photo and comment on an item.
4. The code shows three distinct uses of the YouCam Clothes VTO API (single item, full look, swap re-render) plus at least one accessory API (bag or hat) if time allows.
5. The repo runs locally from a README in under 10 minutes with a YouCam key and the included 500-SKU dataset.

Non-goals (say no early so the build finishes)

- No real payment or checkout. "Add to cart" writes to the host store's cart via a stub adapter; the demo stops at the cart drawer.
- No Skin AI, makeup or hair APIs.
- No size recommendation or body measurement capture (the VTO model handles fit visually; height/weight are only asked if a chosen API needs them, which the Clothes API does not).
- No full in-app wardrobe upload flow in the widget. Wardrobe is shown through the existing Elara app in the video and referenced as the data source for "from your closet" swap options.
- No multi-store account system. Style DNA persists via a signed browser token plus optional email, not a login wall.
- No mobile native app; responsive web only (the widget must look right on a phone-width viewport because many judges watch on one).

Scope tiers

Tier	Features	Rule
P0: must ship	Widget embed, onboarding (3 questions + photo), Style DNA picker (4 looks), chat agent with clarification, outfit cards with try-on, single-item swap, add to cart stub, demo store with 500 SKUs, demo video	Everything in the video must be P0

Tier	Features	Rule
P1: ship if P0 is done by Sunday 6 PM IST	Compatibility score with explanation, Studio tab (browse + multi-select try-on), Style Crew share link + join + per-item thread, "why this works" panel	Each P1 item is self-contained and can be cut without breaking P0
P2: stretch / video-only	Crew group photo with scene background, accessory try-on (bag, hat) via YouCam accessory APIs, wardrobe-sourced swap options, cross-store taste graph visualisation	Show in video as a mock or from Elara if not built

7. End-to-end user journey (demo happy path)

The demo follows Priya from a product page to a shared cart in nine steps. Each step names the screen, what the user does, what the system does, and the YouCam call if any. This is also the shot list for the video (Section 21).

1. **Store page → widget launcher.** Priya is browsing the demo store. A floating "Style me" button sits bottom-right; a product page also shows a small "Try on" chip on every product card. She clicks "Style me".
2. **Onboarding, screen 1: who and what.** Three questions on one screen: *Who is this for?* (Me / Someone else), *Occasion* (free text + chips: beach trip, wedding, office, date, everyday), *Budget* (slider in store currency, default unset). If "Someone else" she adds relation and age range; gender is asked only when the store sells more than one gender and it cannot be inferred. Profession and age range are optional chips below the fold.
3. **Onboarding, screen 2: photo.** One full-body photo: upload, camera, or "use a sample model" for people who don't want to upload. Guidance text: plain background, full body visible, face unobstructed (these are the YouCam input requirements). The photo is uploaded once to YouCam's File API and the returned `file_id` is cached for the session.
4. **Style DNA picker (cold start).** The system renders four looks on her photo in four different vibes drawn from the store's catalogue (for a beach brief: breezy linen, bold print, sporty, minimal). Four tiles, each with a heart and a cross. Her picks seed the taste profile. *YouCam: 4 × Clothes VTO, `garment_category: full_body` or `auto`, run in parallel while the picker screen shows a progress state.*

5. **Chat.** She lands in the Chat tab with a first message already composed from her answers: *"Goa beach trip, ₹4,000, you liked breezy and minimal."* The agent replies with two or three complete outfits, each a card: look name ("Sunset Linen"), rendered try-on of the full look, piece list with prices and total, "Why this works" accordion, buttons Try on / Swap / Add to cart. If the request is ambiguous it asks one clarifying question first, never more than one per turn. *YouCam: one full-look render per outfit card.*
6. **Swap a piece.** On the "Sunset Linen" card she taps the swap icon beside the trousers. A drawer shows four alternative bottoms from the catalogue ranked by compatibility with the rest of the look, each with a score (e.g. 92) and a one-line reason. She picks one; only the lower body re-renders. *YouCam: Clothes VTO with `garment_category: lower_body` on the already-rendered image as `src`, so the top is preserved.*
7. **Share with Crew.** She taps "Share with Crew". The system creates a crew, generates a link, and composes a share image (her try-on plus product links). She sends it over WhatsApp. Her friend opens it, sees the look, uploads her own photo (optional), and the same outfit is rendered on her. Each piece has a thread; the friend writes "the trousers are a yes, swap the top for the white one". Priya sees the comment inside the widget.
8. **Group photo (P2).** Priya taps "Group photo". The system composes both rendered figures on a beach background matching the occasion. *YouCam: AI Photo Background Change on the composed image, or a pre-rendered scene if credits are tight.*
9. **Add to cart.** She taps "Add to cart" on the final look; the store's cart drawer opens with all pieces. The session's choices (liked, swapped, rejected, bought) are written to her Style DNA graph, which the Elara app shows as her constellation. End of demo.

Gifting branch (shown once in the video, 10 seconds). Arjun types "college farewell outfit for my brother, he's 20". The agent infers a male recipient, skips the gender question, asks only for budget, and returns a men's look. No photo is needed for the recipient: the look renders on a sample model or Arjun can upload his brother's photo with consent.

8. Feature 1: Store widget and onboarding

The widget is a single `<script>` tag that mounts a launcher, a product-card chip and a full-height panel (the TALA screenshots show the target look: left-side panel, Chat/Studio toggle, store branding). Onboarding must finish in two screens and under 45 seconds.

8.1 Launcher and entry points

Entry point	Where	Behaviour
Floating button "Style me"	Bottom-right of every store page	Opens the panel on the Chat tab; first visit opens onboarding
Product-card chip "Try on"	Overlay on each product image in listing and PDP	Opens the panel with that product pre-loaded; if no photo yet, runs the photo step first, then renders that item
Shared crew link `?crew=<id>`	Any store URL	Opens the panel on the Crew view for that crew
Deep link `?look=<id>`	Any store URL	Opens the panel on that saved look (used for share images)

8.2 Onboarding screen 1: who, occasion, budget

- *Who is this for?* Toggle: Me / Someone else. Default: Me. If Someone else: relation (free text or chips: partner, brother, sister, friend, parent, child), age range chips (13–17 is allowed only for viewing recommendations; try-on on a minor's photo is disabled). Gender is asked only if (a) the store catalogue has more than one gender and (b) relation does not imply it.
- *Occasion.* Free text with chips generated from the store's category tags (beach, wedding, office, date night, everyday, festival, gym). One required.
- *Budget.* Slider in store currency with "No limit" checkbox. Stored as max per look, not per item. Optional.
- *Optional, collapsed:* profession, age range for self, preferred fit (relaxed / regular / fitted). These appear as "Help me understand you better (optional)". Removed from the required path per the planning call.
- Removed outright: sizing inputs, height/weight, preferred colours. Colours are learned from Style DNA picks instead.

8.3 Onboarding screen 2: photo

- Three options: upload from device, take with camera, use a sample model (four stock models across skin tones and body types, licensed/royalty-free).
- Client-side checks before upload: image is JPEG/PNG, under 10 MB, long side ⩽ 4096 px, a person is detected (lightweight pose check in browser or a quick server call). Show inline guidance if the check fails: "Step back so your whole body is visible".
- Consent copy under the button: "Your photo is used only to show you clothes and is deleted when you close this session unless you save your profile."

- Upload once to YouCam File API (`POST /s2s/v2.0/file/cloth`), store `src_file_id` in session. Never re-upload the same photo.
- Selfie-only step is removed. One full-body photo serves try-on and Style DNA.

8.4 Style DNA picker (cold start)

- Purpose statement shown to the user: "Pick what feels like you. This teaches your stylist before you've bought anything."
- The agent selects four looks from the catalogue that span distinct vibes relevant to the stated occasion (vibe taxonomy: minimal, bold, classic, street, boho, sporty, romantic, edgy). Each look is rendered on the user's photo via Clothes VTO `full_body` .
- UI: 2×2 grid of rendered tiles, each with heart / cross; a skip link "Just start chatting". Picks are stored as vibe weights (+1 heart, −1 cross) plus colour and silhouette attributes extracted from the picked SKUs.
- Progress: tiles fill in as each render completes (renders run in parallel); a skeleton is shown for pending tiles. Target: first tile within 8 s, all four within 20 s.
- Disliked vibes are down-weighted in retrieval, not banned. A dislike halves that vibe's weight; two dislikes of the same vibe across sessions move it to the "Never list" the Elara app already shows.

8.5 Returning user

If a Style DNA token exists in the browser (or the user enters the email they used before), onboarding is skipped and the agent opens with "Welcome back. Still shopping for the Goa trip, or something new?" using the stored plans.

9. Feature 2: Chat, the agentic outfit recommender

The chat turns one sentence of intent into complete, priced, rendered outfits from this store's catalogue, asking at most one clarifying question per turn. It is a multi-agent pipeline with a single orchestrator so the UI only ever talks to one endpoint.

9.1 Agent roles

Agent	Input	Output	Model / method
Orchestrator	User message, session state (profile, Style DNA, cart, previous looks)	Plan: which agents to call, whether to clarify	LLM with tool calling (Claude Sonnet 4.6 or GPT-class model; the team's existing Elara stack)

Agent	Input	Output	Model / method
Intent parser	Free text	Structured brief: `{for: self/other, recipient:{gender?, age?, relation?}, occasion, vibe[], budget_max, must_include[], exclude[], weather?}`	LLM, JSON-only output, validated against schema
Clarifier	Brief with missing required fields	Zero or one question, with 3–4 chip answers	Rule: required fields are occasion and (if multi-gender store and not inferable) gender; budget is asked only if the user's phrasing implies a constraint ("cheap", "splurge") but gives no number
Retriever	Brief, Style DNA weights	Candidate pool: top 40 SKUs per slot (top, bottom, layer, footwear, accessory)	Hybrid: structured filters (gender, category, price, in-stock) + embedding similarity over product title/description/vibe tags; Style DNA re-ranks
Composer	Candidate pool	2–3 complete outfits, each with slots filled, a name, a total price ⩽ budget, and a reason	LLM selects combinations with constraints: colour harmony rules, one statement piece max, footwear only if the store sells it; rejects any outfit over budget
Compatibility scorer	An outfit or an item + the rest of a look	Score 0–100 and a one-line reason	Weighted rule score (colour harmony 35%, formality match 25%, silhouette balance 20%, Style DNA fit 20%), explanation phrased by the LLM. See Section 11

Agent	Input	Output	Model / method
Renderer	Outfit + `src_file_id`	Try-on image(s)	YouCam Clothes VTO; see Section 15
Memory writer	Every user action	Graph edges (liked, disliked, swapped_out, tried, bought)	Writes to the taste graph (Section 13)

9.2 Conversation rules

- The first agent message after onboarding is proactive and uses what was collected: it never re-asks occasion or budget.
- Maximum one clarifying question per turn. If two things are missing, ask the one that changes the catalogue filter most (gender before budget).
- When shopping for someone else, every pronoun in agent copy refers to the recipient ("for him"), and the try-on uses a sample model of the recipient's gender unless the user uploads the recipient's photo.
- Outfit = top + bottom at minimum (or one-piece: dress, jumpsuit, set). Footwear only if the store carries footwear; the same rule for bags, hats and jewellery.
- Each outfit card shows: name, rendered full-look image (or skeleton while rendering), piece list with thumbnails, size availability, total, "Why this works" accordion, buttons: Try on (if not auto-rendered), Swap (per piece), Add to cart, Share with Crew.
- Streaming status lines while working, in the style of the TALA screenshot: Reading your message → Understanding what you're looking for → Searching the catalogue → Styling your looks → Rendering on you. Each ticks as the stage completes.
- Follow-up requests modify the current look rather than restarting: "make it more formal", "something under ₹3,000", "show me in blue" are applied to the selected look and re-rendered.
- Attachments: the paperclip accepts an inspiration image ("something like this"). It is embedded and used as a retrieval query; it is not sent to YouCam.

9.3 Inference of gender and recipient

Recipient gender is inferred from relation words (brother, boyfriend, dad → male; sister, girlfriend, mum → female) and explicit pronouns. If the store is single-gender, the field is set from the store config and never asked. If the store is multi-gender and nothing can be inferred for the shopper themselves, the clarifier asks once with chips: Women / Men / Show me both.

9.4 Budget handling

Budget is a cap on the full look. The Composer must return looks at or under the cap; it may add a cheaper fourth look labelled "Under budget pick". If no combination fits, the agent says so and offers the closest look with the overage stated.

9.5 Failure and fallback

- Catalogue returns nothing for a slot → the agent drops the slot and says what it could not find rather than hallucinating a product.
- Render fails or times out (>45 s) → card shows the product flat-lay collage with a "Retry try-on" button; the conversation continues.
- LLM returns invalid JSON → one automatic retry with the schema error appended; then a safe default response.

10. Feature 3: Studio, manual outfit building (P1)

Studio is the browse-and-compose counterpart to Chat for shoppers who prefer to pick: a filtered catalogue grid, a tray of selected pieces, and one Try on / Add to cart action for the whole set. The TALA Studio screenshot is the reference layout.

10.1 Layout

- Top: gender toggle (hidden for single-gender stores), category tabs generated from the catalogue (Tops, Bottoms, Dresses & Sets, Layers, Bags, Accessories, Activewear…), Filters button (price, colour, size, vibe).
- Grid: 3 columns on desktop, 2 on mobile. Card: image, name, price, select checkmark on hover/tap, small "Try on" chip.
- Bottom tray: thumbnails of selected pieces with remove ×, running total, buttons Try on and Add to cart. Tray enforces one item per slot (selecting a second top replaces the first, with an undo toast).
- "Ask the stylist" link on the tray sends the selection to Chat as context: "Finish this look for me" → Composer fills the empty slots.

10.2 Behaviour

- Try on with 1 item → single-item render on the user photo (`garment_category` by slot).
- Try on with 2+ items → sequential layered render: upper then lower (or full-body if a one-piece), each call using the previous result as `src`. Result opens in the Try-on viewer (Section 11) with swap handles on each piece.
- Studio selections write to the taste graph as `considered` edges (weaker than `liked`).
- Empty state: "Pick a few pieces and I'll show them on you" with three suggested starter pieces from Style DNA.

10.3 Out of scope for Studio in the demo

Size selection, stock by size, wishlists and sort-by-popularity. The Studio exists to show the second interface promised in the call; the video spends at most 15 seconds on it.

11. Feature 4: Virtual try-on, swap and compatibility score

Try-on is the way every recommendation is presented, so it must work in three modes: a single item on the user, a full look, and a swap that re-renders only one slot of an existing result. All three use the YouCam Clothes VTO API with different `garment_category` and `src` choices.

11.1 Render modes

Mode	Trigger	`src` image	`ref` image	`garment_category`	Calls
Single item	"Try on" chip on a product card	User photo `src_file_id`	Product image	By slot: `upper_body`, `lower_body`, `shoes`; `auto` if slot unknown	1
Full look (one-piece)	Outfit card with dress/jumpsuit/set	User photo	Product image	`full_body`	1
Full look (separates)	Outfit card with top + bottom (+ layer)	User photo → result of step 1 → …	Each piece's image in order: bottom, top, layer	`lower_body`, then `upper_body`, then `upper_body` for the layer	2–3, sequential
Swap one piece	Swap handle on a rendered look	Current rendered result	New piece's image	That slot's category	1
Style DNA tiles	Onboarding	User photo	A representative full-body product image per vibe, or composed look	`full_body` / `auto`	4, parallel

Order for separates: render the bottom first, then the top, so the top's hem and any tuck sit naturally over the trousers. `change_shoes` is set to `false` on `lower_body` calls unless the look includes footwear from the store, so the shopper's own shoes stay in the picture.

11.2 Try-on viewer

- Full-height image with a before/after slider (the shopper's original vs the render), modelled on the TALA before/after screenshot.
- Hotspot per rendered piece (top, bottom, layer, footwear), each with a small swap icon. Tapping opens the Swap drawer for that slot.

- Below the image: piece list with prices, compatibility score badge for the whole look, Add to cart, Share with Crew, Save look, Download image.
- Loading: skeleton silhouette with the status line "Rendering on you… ~15 s". Progress is real (polling), not a fake bar.
- Error: "We couldn't render this one. Try another photo or a different piece." with the flat product image shown so the card is never empty.

11.3 Swap drawer

- Opens from the right (desktop) or bottom (mobile). Title: "Swap the trousers".
- Shows 4–6 alternatives for that slot from the catalogue, ranked by compatibility with the *other* pieces of the current look, filtered to the remaining budget. Each tile: product image, name, price, compatibility score (e.g. 92), one-line reason ("Same muted palette, straighter cut balances the loose top").
- Source toggle (P2): Store / Your wardrobe. Wardrobe items come from the Elara wardrobe API and render the same way, but have no price and no Add to cart.
- Selecting a tile re-renders only that slot using the current result as `src`. The previous render is kept in a history strip so the shopper can step back.
- The swapped-out item is written to the taste graph as `swapped_out` with the slot and the chosen replacement, which is the strongest preference signal the system collects.

11.4 Compatibility score

A number from 0 to 100 shown beside each candidate in the Swap drawer and on each outfit card, with a one-line reason. It is computed by rules so it is fast, deterministic and explainable; the LLM only phrases the reason.

Component	Weight	How it is scored
Colour harmony	35	Dominant colours per piece (pre-extracted from product images into the catalogue). Score by hue relationship: neutral + anything = high; analogous or complementary = high; clashing warm/cool saturated pairs = low
Formality match	25	Each SKU carries a formality level 1–5 (tagged at ingestion). Score falls with the gap between pieces and with distance from the occasion's target level
Silhouette balance	20	Loose top + fitted bottom or fitted top + loose bottom scores high; loose + loose or fitted + fitted scores mid; tagged at ingestion (fit: relaxed/regular/fitted)

Component	Weight	How it is scored
Style DNA fit	20	Cosine similarity between the piece's vibe/colour vector and the user's current taste vector

Score bands shown to the user: 85–100 "Great match", 70–84 "Works", below 70 "Bold choice" (never hidden, never called bad). The reason template is: *[strongest component in plain words] + [one trade-off if any component is under 60].*

11.5 Accessory try-on (P2)

If the store sells bags, hats or scarves, the Composer may add one accessory slot and the viewer renders it with the matching YouCam accessory API (AI Bag, AI Hat, AI Scarf, each requiring a `gender` parameter). This is the "at least three capabilities" idea from the planning call; it is shown only if P0 and P1 are complete.

12. Feature 5: Style Crew, group shopping (P1; group photo P2)

Style Crew moves the "should I buy this?" conversation from a WhatsApp screenshot into the store, where friends see the look on the shopper (and on themselves), react per item, and the store captures the moment of decision. It is the feature most likely to make judges remember the entry, so the demo gives it 30 seconds.

12.1 Create and share

1. From any rendered look, "Share with Crew" creates a crew (if none exists for this session) and adds the look to it.
2. The system composes a share image: the try-on render, the look name, piece thumbnails with prices, the store logo, and a QR/short link. Image is 1080×1920 for stories and 1080×1080 for chat.
3. Share sheet: WhatsApp, copy link, download image. Link format `https://<store>/?crew=<crewId>`; the widget intercepts it on load.
4. Crew names default to the occasion ("Goa trip crew"), editable.

12.2 Join

- Recipient opens the link; the widget opens on the Crew view with the shared look. No login: a display name and an optional photo. If they upload a photo, every look in the crew is rendered on them too ("See it on you").
- Crew size for the demo: 2–4 members. Members are stored by display name + anonymous id; no accounts.

12.3 Inside the crew

- Look feed: each shared look is a card showing the render on each member who has a photo (horizontal strip), the piece list, and the total.
- Per-item thread: every piece has its own comment thread and a quick reaction (thumbs up / down / "swap it"). Thread counts show as badges on the piece thumbnails. A "swap it" reaction from a friend opens the owner's Swap drawer pre-filtered to the friend's suggestion if they named one.
- Votes roll up to a crew verdict per look ("3 of 4 love it").
- The look owner can apply a friend's suggested swap with one tap; the crew sees the new render.
- Realtime: Supabase Realtime (or Firebase) channel per crew for comments, reactions and new renders. Polling every 5 s is the acceptable fallback.

12.4 Group photo (P2)

- Button "Group photo" when 2+ members have renders. The system composes the rendered figures side by side (background removed with AI Photo Background Removal, or client-side cut-out from the VTO result if the source photos have plain backgrounds), then places them on a scene matching the crew's occasion via AI Photo Background Change (prompt: "sunny beach in Goa, golden hour").
- Output is a shareable image with the store logo; it is the natural end of the video.
- Credit note: this is two or three extra YouCam calls per group photo; disable it if the budget in Section 15 is under 100 units.

12.5 Taste graph effect

Friend reactions write weak edges to the owner's graph (`friend_liked`, `friend_disliked`) so the agent can say "your crew preferred the white top last time". Friend's own picks write to the friend's graph, which is how a Crew member becomes a shopper with a warm start.

13. Feature 6: Taste graph and Style DNA memory

Every action the shopper takes becomes an edge in a personal graph that is keyed to the shopper, not the store, so the agent gets smarter on the first visit and carries that knowledge to the next store that runs the widget. The Elara app already renders this graph as the "Style Constellation" (Overview, Style Map, Style Play, Timeline); the widget writes to the same structure.

13.1 Nodes

Node type	Examples	Source
User	Priya	Session token or email
Plan / occasion	Goa trip 10–14 Nov, college farewell	Onboarding, chat
Vibe	minimal, boho, street	Style DNA picks, liked outfits
Attribute	colour: sage, fit: relaxed, formality: 2	Extracted from liked/disliked SKUs
Product	SKU 1234 (store-scoped)	Catalogue
Wardrobe item	Blue denim zip-up jacket	Elara wardrobe
Person (recipient)	brother, 20, male	"Someone else" flow
Crew	Goa trip crew	Style Crew

13.2 Edges and weights

Edge	Written when	Weight
`hearted` / `crossed`	Style DNA tile	+1.0 / −1.0 on vibe and attributes
`liked_outfit`	Thumbs-up on an outfit card	+0.8
`tried_on`	Any render	+0.3 (considered)
`swapped_out` → `swapped_in`	Swap drawer selection	−0.7 on the out item's distinguishing attributes, +0.7 on the in item's
`added_to_cart`	Cart action	+1.0
`friend_liked` / `friend_disliked`	Crew reaction	±0.3
`never`	Two crosses on the same vibe or an explicit "never show me X"	Hard filter
`shopping_for`	Recipient profile	Scopes the above to that person

13.3 How the agent uses it

- The Retriever re-ranks candidates by cosine similarity to the user's taste vector (sum of weighted attribute edges, decayed 10% per month).
- The Composer reads `never` edges as hard excludes and `plan` nodes to pre-fill occasion on return visits.
- The Clarifier skips any question the graph already answers.
- A recipient's sub-graph is used when `shopping_for` is set, so a shopper's own taste does not leak into a gift.

13.4 Storage

Postgres with a `nodes` and `edges` table (JSONB attributes) is enough for the demo; Neo4j is unnecessary. The Elara app reads the same tables to draw the constellation. Identity: a signed token in `localStorage` for anonymous sessions; entering an email merges the anonymous graph into the known one.

13.5 Privacy

Photos are not stored in the graph; only `file_id` references with a 24-hour TTL (matching YouCam's task-result validity) unless the shopper saves their profile. Graph export and delete are one API call each, exposed as "Download my style data" / "Forget me" links in the widget footer.

14. Feature 7: Cart handoff and plug-and-play integration

A store integrates in two steps: paste one script tag and point us at a product feed. Nothing in the widget is hard-coded to a catalogue, which is the claim the demo store exists to prove.

14.1 Embed

```
<script src="https://cdn.elara.shop/widget.js"
        data-store-id="demo-northline"
        data-theme="auto"
        data-position="bottom-right"
        data-cart-adapter="shopify"></script>
```

- `widget.js` mounts a Shadow DOM root so store CSS cannot break it and ours cannot leak.
- `data-theme` reads the store's primary colour and font from CSS variables if present (`--brand-primary` , `--brand-font`), else uses our neutral theme. The TALA screenshots show the intended result: the panel looks native to the store.

- `data-cart-adapter`: `shopify` (uses `/cart/add.js`), `woocommerce`, or `custom` (store provides `window.elaraAddToCart(items)`). The demo store uses `custom` with a visible cart drawer.

14.2 Product feed

The store exposes a JSON or CSV feed (or a Shopify product export). Required fields and the enrichment we add at ingestion:

Field	Required	Added by ingestion
`id`, `title`, `price`, `currency`, `image_url`, `product_url`	Yes	—
`gender` (women/men/unisex)	Yes	—
`category` (top/bottom/dress/set/layer/footwear/bag/accessory)	Yes, or mapped from store categories	Slot mapping
`description`, `sizes_in_stock`, `colour`	Recommended	Dominant colours extracted from image if `colour` missing
`vibe_tags[]`, `formality` 1–5, `fit`	No	LLM-tagged from title + description + image, once per SKU
`embedding`	No	Text embedding of title + description + tags
`vto_ready`	No	Image checked for a clean single-garment view suitable as a VTO `ref`

Ingestion runs once per feed update (`scripts/ingest.ts`) and writes to Postgres + pgvector. For the demo it runs on the 500-SKU dataset Saksham provides (Section 17).

14.3 Add to cart

- Outfit-level: one button adds every piece in its default size (or the size the user picked in the piece list). Pieces out of stock are shown struck through with "Notify me".
- The widget calls the adapter, then opens the store's cart drawer and writes `added_to_cart` edges to the graph.
- No checkout is built. The video ends on the cart drawer with the total.

14.4 Store dashboard (video-only mention)

A single analytics page is described, not built: try-ons per session, swap-outs by SKU (which products get rejected), crew conversions, top vibes per store. This is the merchant-value story for the "Potential impact" criterion; one slide in the video is enough.

15. YouCam API integration plan

The primary API is **AI Clothes Virtual Try-On** (`POST /s2s/v2.0/task/cloth-v3` , or `cloth-v4` if the v4 engine is enabled on the hackathon key), called server-side through a thin wrapper with caching and a credit budget. All details below are taken from the YouCam docs (AI Clothes v3 reference, cloth-v4 task, MCP capability list, changelog); confirm the per-call unit cost in the API console (`Get_Feature_Cost`) before Saturday night because it decides the credit budget.

15.1 APIs used

API	Endpoint	Used for	Priority
File upload	`POST /s2s/v2.0/file/cloth` (or `src_file_url` with a public URL)	User photo once per session; product images are passed as `ref_file_url` from our CDN	P0
AI Clothes VTO	`POST /s2s/v2.0/task/cloth-v3` → `GET /s2s/v2.0/task/cloth-v3/{task_id}`	Single item, full look, swap, Style DNA tiles	P0
AI Bag / AI Hat / AI Scarf VTO	`/s2s/v2.0/task/bag` , `/hat` , `/scarf` (each needs `gender`)	Accessory slot in outfits	P2
AI Photo Background Change	`/s2s/v2.0/task/background-change` (or template)	Crew group photo scene	P2

API	Endpoint	Used for	Priority
AI Photo Background Removal	`/s2s/v2.0/task/background-removal`	Cut-outs for group photo composition	P2

15.2 Request contract (Clothes VTO)

```
POST https://yce-api-01.makeupar.com/s2s/v2.0/task/cloth-v3
Authorization: Bearer <YOUCAM_API_KEY>
{
  "src_file_id": "<user photo file id>",        // or src_file_url
  "ref_file_url": "https://cdn.../sku-1234.jpg",  // product image
  "garment_category": "lower_body",             // full_body | upper_body |
lower_body | shoes | auto
  "change_shoes": false                           // only matters for
full_body / lower_body
}
→ { "status": 200, "data": { "task_id": "..." } }
```

Then poll `GET .../task/cloth-v3/{task_id}` at the `polling_interval` the response gives, until `status` is `success` or `error`. The result is valid for 24 hours by `task_id`; download the result image immediately and store it on our side (Supabase Storage / S3) so the UI never depends on YouCam's URL expiring.

15.3 Input requirements enforced client-side

- Source: high-resolution full-body photo, whole body visible, face unobstructed, JPEG/PNG under 10 MB, long side ⩽ 4096 px.
- Reference: product image of the garment alone (flat-lay or on-model both accepted per docs); we prefer the flat/ghost image when the catalogue has one, flagged `vto_ready` at ingestion.

15.4 Wrapper behaviour (`services/youcam.ts`)

- Single function `tryOn({src, ref, category, changeShoes})` returning `{imageUrl, taskId, unitsUsed}`.
- **Cache key** = hash(src_file_id, ref SKU id, category, changeShoes). Identical requests (e.g. two crew members viewing the same look, or a judge re-clicking) return the cached image at zero cost.
- **Concurrency**: at most 4 tasks in flight per session (Style DNA tiles) and 8 per server; a queue with backoff on HTTP 429.
- **Timeout**: 60 s hard; show fallback (flat product image) after 45 s but keep polling in the background and swap the image in if it arrives.

- **Errors**: `CreditInsufficiency` → switch the whole app to "demo mode" with pre-rendered images and a banner; `InvalidParameters` → log the category and retry once with `auto`; `429` → exponential backoff 2 s / 4 s / 8 s.
- **Webhook**: not used; polling is simpler to run locally for judges.

15.5 Credit budget (1,000 units per registered participant; two participants = 2,000)

Per-call cost must be read from the console; the plan below assumes **X units per Clothes VTO call** and should be re-done the moment X is known.

Use	Calls per run	Runs	Calls
Development and debugging	—	—	~60
Pre-rendering the Style DNA tiles for 4 sample models × 8 vibes	32	1	32
Pre-rendering every outfit in the 12 scripted demo looks (full look = 2 calls)	24	1	24
Video recording takes	~15	3	45
Judges testing live (estimate 10 judges × 6 renders)	6	10	60
Buffer	—	—	40
Total			**~260 calls**

If X ⩽ 7 this fits in 2,000 units with room; if X is higher, cut live Style DNA (serve pre-rendered tiles for sample models, live only for uploaded photos) and reduce judge allowance. The "Share & Earn Bonus" on the hackathon page gives extra units for posting the project publicly; do it on Saturday.

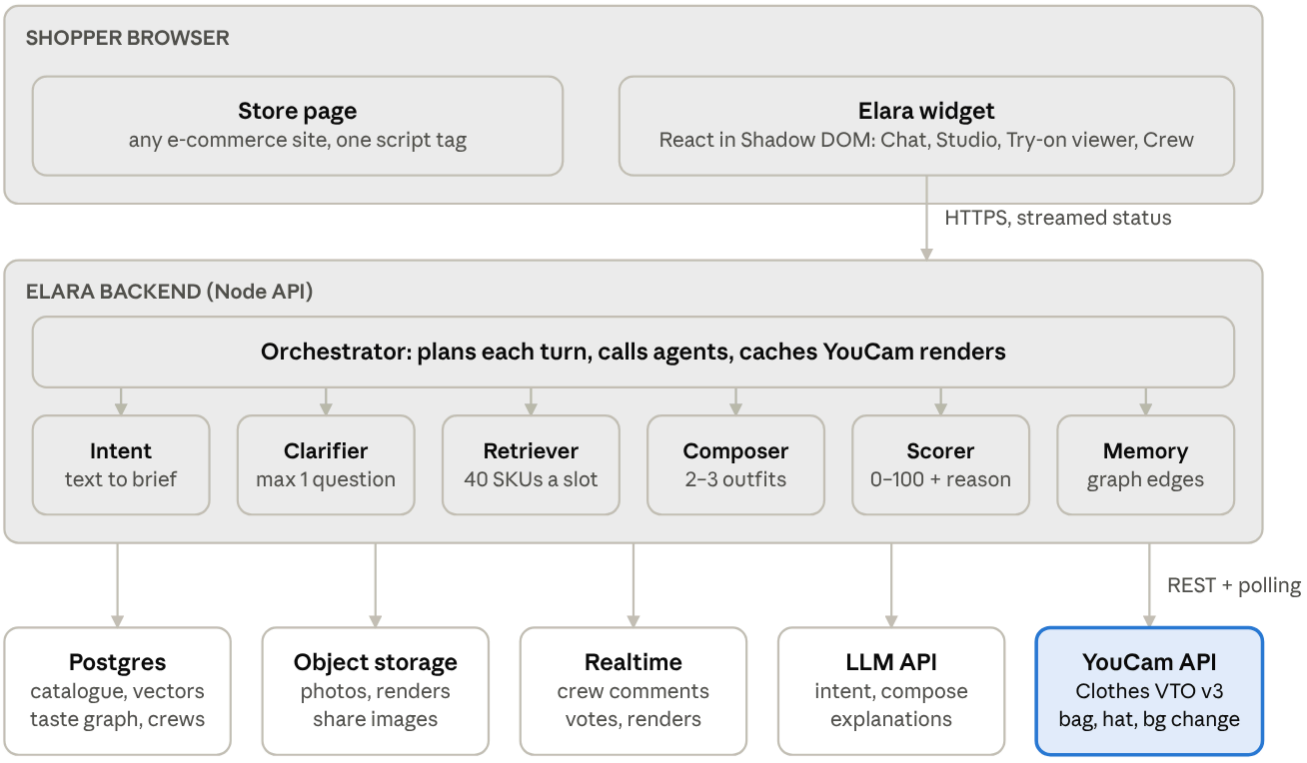
15.6 Demo-mode fallback

Every render the video needs is pre-generated and stored with its request hash. If the key runs out or the API is slow during judging, the wrapper serves the stored image for a matching hash and shows a small "cached" tag. This protects the demo without faking anything: every image was produced by the YouCam API.

16. System architecture and tech stack

Three layers: a React widget in the shopper's browser, one Node backend that runs the agent pipeline, and five services behind it, of which the YouCam API is the one that turns every recommendation into a picture.

One widget and one backend; the YouCam API renders every recommendation



system architecture · widget, backend agents, services

The widget never calls YouCam or the LLM directly; all keys stay on the backend, and every render goes through the cache in the orchestrator so repeated requests cost nothing.

Tech stack (chosen for a 36-hour build by two people who already run Elara)

Layer	Choice	Why
Widget	React 18 + Vite, TypeScript, Tailwind, mounted in Shadow DOM; bundled as `widget.js`	Saksham's existing stack; Shadow DOM isolates store CSS
Demo store	Next.js static site with 500 SKUs, fictional brand, `custom` cart adapter	Judges need a store to click; Next gives hosting on Vercel in minutes
Backend	Node 20, Next.js API routes or Fastify, TypeScript	Same repo, same language; streaming via Server-Sent Events for status lines
Agents	LLM via API with tool calling and JSON schema outputs; prompts in `agents/*.md`	Deterministic structure, easy to show judges

Layer	Choice	Why
Catalogue and graph	Supabase Postgres + pgvector; tables `products`, `nodes`, `edges`, `crews`, `looks`, `renders`	One managed DB, Realtime channels included for Crew
Storage	Supabase Storage (or S3) for user photos, renders, share images	Renders must outlive YouCam's 24-hour task window
YouCam	`services/youcam.ts` wrapper: upload, cloth-v3 task, poll, cache, budget	Section 15
Image composition	`sharp` on the server for share images and group-photo collage; Satori or Canvas for text overlays	No client-side canvas fiddling
Hosting	Vercel (store + API), Supabase (DB, storage, realtime)	Free tiers, zero ops
Repo	Monorepo `apps/widget`, `apps/store`, `apps/api`, `packages/agents`, `packages/catalog`, `scripts/ingest.ts`, `data/products.json`	One README runs everything with `pnpm dev`

Request flow for one chat turn

1. Widget POSTs `{sessionId, message}` to `/api/chat` and opens an SSE stream.
2. Orchestrator loads session state and taste vector, runs Intent → Clarifier. If a question is needed it streams it and stops.
3. Retriever queries pgvector with filters; Composer builds 2–3 outfits; Scorer attaches scores and reasons. The outfit cards stream to the widget with `renderStatus: pending`.
4. Orchestrator queues renders through the YouCam wrapper (cache first); each completed image streams as a card update.
5. Memory writer records `tried_on` edges. Stream closes.

17. Data model and the 500-SKU demo catalogue

Six tables carry the whole demo; the catalogue is the one Saksham must deliver first because ingestion, retrieval and pre-rendering all wait on it.

17.1 Tables

Table	Key columns	Notes
`products`	`id`, `store_id`, `title`, `description`, `price`, `currency`, `gender`, `category`, `slot`, `colours[]`, `vibe_tags[]`, `formality` 1–5, `fit`, `sizes_in_stock[]`, `image_url`, `vto_ref_url`, `product_url`, `embedding vector(1536)`, `vto_ready`	`vto_ref_url` is the clean garment image used as YouCam `ref`; defaults to `image_url`
`sessions`	`id`, `store_id`, `user_token`, `src_file_id`, `src_photo_url`, `brief jsonb`, `shopping_for jsonb`, `created_at`, `expires_at`	One per widget open; `src_file_id` is the uploaded user photo
`looks`	`id`, `session_id`, `name`, `items jsonb[]` (slot, product_id, size), `total`, `score`, `reason`, `render_id`, `parent_look_id`	`parent_look_id` links a swap to the look it came from
`renders`	`id`, `cache_key`, `src_file_id`, `ref_product_id`, `garment_category`, `change_shoes`, `task_id`, `status`, `image_url`, `units_used`, `created_at`	`cache_key` unique; this table is the YouCam cache
`nodes` / `edges`	`nodes(id, user_token, type, attrs jsonb)`, `edges(id, from_node, to_node, type, weight, created_at)`	Taste graph, Section 13
`crews` / `crew_members` / `crew_comments`	`crews(id, name, occasion, owner_token)`, `crew_members(crew_id, display_name, token, src_file_id)`, `crew_comments(id, crew_id, look_id, slot, author, text, reaction, created_at)`	Realtime publication on `crew_comments`

17.2 Demo catalogue requirements (owner: Saksham)

- ~500 SKUs across women's and men's wear so the gifting flow works. Suggested split: 220 women, 220 men, 60 unisex/accessories.
- Slot coverage per gender: tops 70, bottoms 50, dresses/sets 30 (women), layers 25, footwear 20, bags 10, accessories 15. Without footwear and bags in the feed the agent will correctly omit those slots, so include some.
- Each SKU needs: title, price in INR, product image URL that is publicly reachable (YouCam fetches `ref_file_url` itself), gender, store category. Everything else is generated at ingestion.
- Source: Myntra, Zara or H&M listings. Strip brand names from titles for the demo store; keep images. If a listing's only image is on-model with a busy background, mark `vto_ready=false` and the Retriever will prefer other SKUs for rendering.
- Format: `data/products.json` array, or a CSV with the same headers. A 10-row sample before the full file lets ingestion be written in parallel.

17.3 Ingestion (`scripts/ingest.ts`)

1. Validate rows against the schema; reject rows with no image or price.
2. Map store category to `slot` with a lookup table; unknown categories go to a review list.
3. For each SKU call the LLM once with title + description + image to produce `vibe_tags`, `formality`, `fit`, `colours` (JSON schema). Batch 20 per call; ~25 calls for 500 SKUs.
4. Embed title + description + tags; write to pgvector.
5. Flag `vto_ready` by a quick image check (single garment, mostly plain background).
6. Pre-render the 32 Style DNA tiles for the four sample models and the 12 scripted demo looks (Section 15.5), writing to `renders`.

17.4 Scripted demo looks

Twelve looks are chosen by hand from the catalogue and listed in `data/demo-looks.json`: four beach (women), four farewell (men), two office, two date night. Each has a name, pieces, a target total and a "why this works" line written by the team so the video copy is controlled. The agent is still live; these are the looks it is steered towards for the scripted prompts by seeding them as high-weight `liked_outfit` edges on the demo profile.

18. UI/UX specification

The panel is 480 px wide on desktop (full-screen on mobile), slides in from the right over the store, and inherits the store's primary colour and font. The TALA screenshots are the visual reference: large type, black pill buttons, white surfaces, generous spacing, status lines with check marks.

18.1 Screen inventory

#	Screen	Key elements	States
1	Launcher	Floating pill "Style me" with sparkle icon; product-card chip "Try on"	Idle, hover, badge when crew has new comments
2	Onboarding 1	Store logo, "Who is this for?" toggle, occasion chips + input, budget slider, optional accordion, Continue	Validation: occasion required
3	Onboarding 2	Photo drop zone with camera and sample-model options, guidance illustration, consent line, Continue	Uploading, checking, rejected (reason), done
4	Style DNA picker	2×2 rendered tiles with heart/cross, progress text, Skip link	Rendering (skeleton per tile), 2 picked → Continue enabled
5	Chat	Header (logo, Chat/Studio toggle, theme, new chat, close), message list, outfit cards, status checklist, composer with paperclip and send	Streaming, clarifying question with chips, error with retry
6	Outfit card	Number + name, render image (tap to open viewer), piece row (thumb, name, price, swap icon), total, "Why this works" accordion, buttons Try on / Add to cart / Share	Render pending (skeleton), ready, failed (flat collage)
7	Try-on viewer	Full-height image, before/after slider, piece hotspots with swap icons, score badge, action bar, history strip	Rendering, ready, swapping (only that slot shimmers)
8	Swap drawer	Title, source toggle (Store / Wardrobe, P2), 4–6 tiles with score and reason, Apply	Loading, empty ("Nothing else fits the budget; raise it?")
9	Studio	Gender toggle, category tabs, filters, grid, selection tray with Try on / Add to cart	Empty tray, 1 item, 2+ items

#	Screen	Key elements	States
10	Share sheet	Preview of share image, WhatsApp / Copy link / Download	Generating image, ready
11	Crew	Crew name, members strip, look cards with per-member renders, per-item threads, verdict bar, Group photo button	Joining (name + optional photo), live updates
12	Cart confirmation	Toast "3 items added" + store cart drawer opens	—

18.2 Component library

Buttons (primary black pill, secondary outline pill, icon button), chips (selectable, removable), outfit card, product tile, render frame (with skeleton and error variants), score badge (three bands), status checklist, drawer (right/bottom), before/after slider, thread list, toast. All built as small React components in `apps/widget/src/components` with Tailwind tokens read from the store theme.

18.3 Copy and tone

Second person, short, stylist-like, no exclamation marks except the welcome line. The agent never says "as an AI". It names looks ("Sunset Linen"), gives one reason per look, and admits what it could not find. Clarifying questions are one sentence with chips.

18.4 Loading and latency design

- Outfit text arrives first (under 4 s); renders follow one by one. Cards never block on images.
- Status checklist mirrors real pipeline stages and ticks on real events.
- Any wait over 20 s shows a line of copy that does work for the user ("Checking the trousers fit the budget…") so the wait reads as effort, not failure.

18.5 Accessibility and responsiveness

Keyboard-navigable chips and buttons, alt text on renders ("You wearing Sunset Linen: linen shirt and wide trousers"), colour contrast AA, panel full-screen under 640 px with the composer pinned to the bottom and safe-area padding on phones. The video is recorded on desktop with one phone cut-in.

18.6 Student-track UI variant

Same components, different theme: colour scheme, type family, launcher copy, look-card layout (horizontal vs vertical), and a different fictional store (streetwear instead of resort wear). Diverge only in `theme.ts`, the store dataset and the landing page so one codebase ships both.

19. Non-functional requirements

The demo must survive a judge on a slow connection, an expired API key and a photo taken in a messy room. These targets are the ones the build is checked against on Sunday evening.

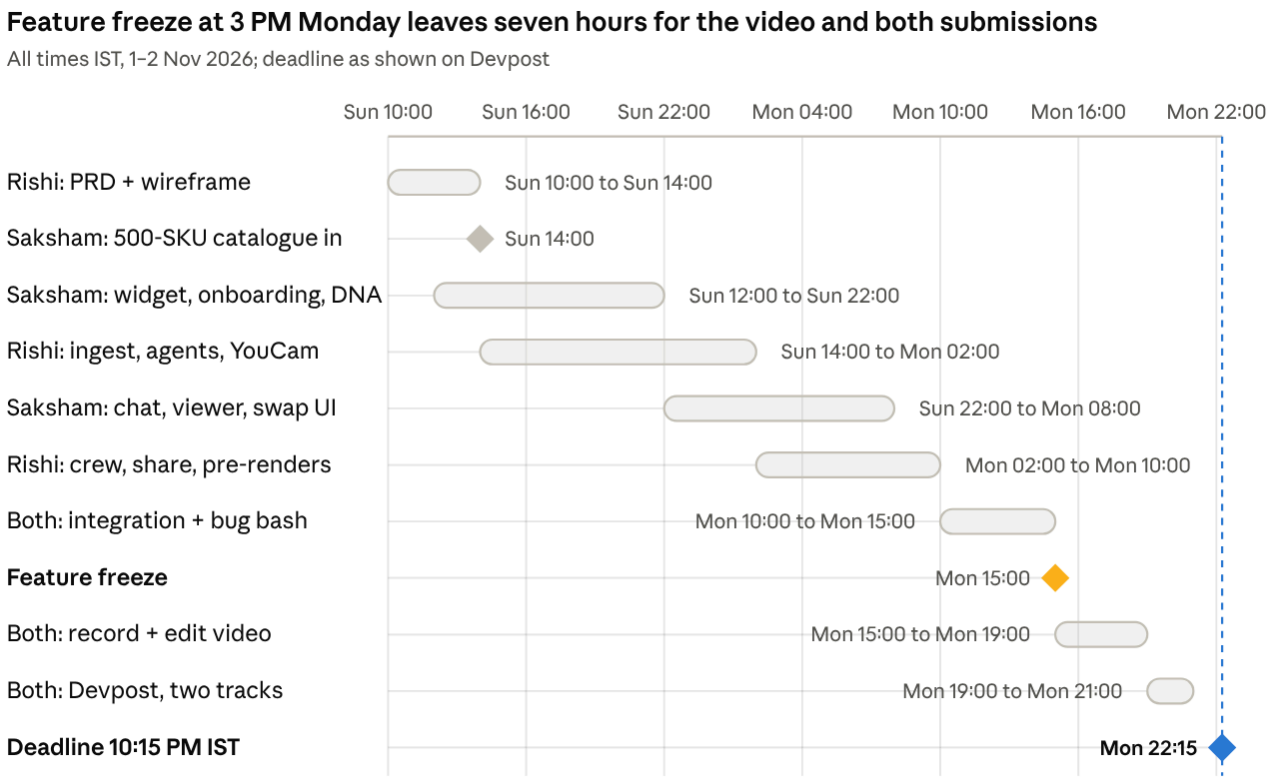
Area	Requirement	How it is met
Latency	First agent text within 4 s; first render within 15 s (p50) and 45 s (p95); swap re-render within 20 s	Streaming text before images; parallel renders; cache; pre-rendered demo looks
Availability during judging	The hosted demo works for at least 48 hours after submission with no team action	Vercel + Supabase free tiers; demo-mode fallback if YouCam credits run out; health check page `/status`
Credit safety	Never exceed the YouCam unit budget; never block the UI on a failed render	Budget counter in `renders`; per-session cap of 20 live renders; fallback to cached images
Error handling	Every failure has copy and a next action; no raw errors reach the UI	Error map in the wrapper; card-level error states; toast for network loss
Security	API keys only on the server; uploaded photos private; crew links unguessable	Env vars; Supabase Storage signed URLs; crew ids are 16-char random tokens
Privacy	Photos deleted at session expiry (24 h) unless the shopper saves a profile; minors never rendered	TTL job; age gate on "someone else" flow; consent copy on upload
Content safety	Reject uploads that are not a clothed full-body photo of a person; refuse try-on of lingerie/swimwear on uploaded photos of others	Lightweight classifier or LLM vision check; category rule in Composer

Area	Requirement	How it is met
Determinism for the video	The scripted prompts produce the scripted looks	Seeded demo profile; temperature 0.2 for Composer; fixed catalogue snapshot
Repo quality	Clone → `.env` → `pnpm i && pnpm dev` runs store + widget + API locally in under 10 min	README with env list, seed script, sample key instructions; MIT licence
Observability	Every YouCam call logged with cache hit/miss, units, latency	`renders` table + console; a `/admin/renders` page to show judges the API usage
Mobile	Panel usable on a 390 px viewport; no horizontal scroll	Responsive layout; tested on iPhone Safari and Android Chrome

Browser support: last two versions of Chrome, Safari, Firefox, Edge. Shadow DOM and `fetch` streaming are required; no IE.

20. Build plan: the next 36 hours

Two parallel lanes until Monday 10 AM, then everything converges on integration, a hard feature freeze at 3 PM, and seven protected hours for the video and the two Devpost submissions.



build timeline · 1–2 Nov IST, by owner

The freeze is the one date that cannot move: a feature that is not working at 3 PM Monday is cut or shown as a mock in the video, never finished during recording.

Ownership

Workstream	Owner	Deliverable	Done when
PRD + wireframe	Rishi	This doc; HTML wireframe of screens 2–8	Shared with Saksham (Sunday 2 PM)
Catalogue	Saksham	`data/products.json`, 500 SKUs, public image URLs	Ingestion runs clean
Ingestion, retrieval, agents	Rishi	`scripts/ingest.ts`, `/api/chat` streaming, agent prompts, scorer	Scripted prompt returns 3 priced outfits in under 4 s
YouCam wrapper	Rishi	`services/youcam.ts` with cache, budget, fallback	Single, full-look and swap renders work on a real photo
Widget + onboarding + Style DNA	Saksham	`apps/widget` screens 1–4, Shadow DOM mount, theme	Embeds on the demo store and completes onboarding

Workstream	Owner	Deliverable	Done when
Chat, outfit cards, viewer, swap drawer	Saksham	Screens 5–8 wired to `/api/chat` and `/api/render`	Swap re-renders one slot live
Studio (P1)	Saksham	Screen 9	Multi-select try-on works
Crew + share (P1)	Rishi	Screens 10–11, Realtime, share image	Second browser joins and comments
Pre-renders	Rishi	32 DNA tiles + 12 demo looks in `renders`	Demo mode serves them with the key disabled
Demo store	Saksham	`apps/store` with fictional brand, cart drawer	Deployed on Vercel
Video	Both	2:45 cut, YouTube public	Uploaded by 8 PM Monday
Devpost	Both	Two submissions, descriptions, screenshots, repo links	Submitted by 9 PM Monday

Checkpoints

- [ ] Sunday 2 PM: catalogue sample (10 rows) and PRD exchanged
- [ ] Sunday 10 PM: first live try-on from the widget on the demo store
- [ ] Monday 2 AM: chat returns outfits with renders end-to-end
- [ ] Monday 10 AM: swap, crew join and share image working; pre-renders done
- [ ] Monday 3 PM: feature freeze; decide P1/P2 cuts; start recording
- [ ] Monday 8 PM: video public on YouTube; repo README tested from a fresh clone
- [ ] Monday 9 PM: both Devpost forms submitted; confirmation emails received

21. Submission package

Judges are not required to watch past three minutes, so the video must show the YouCam API working on a real photo inside the first 40 seconds and the full shopper journey by 2:30. Everything else (repo, description, screenshots) supports what the video shows.

21.1 Checklist (per submission; two submissions)

- [ ] Public GitHub repo with MIT licence, README (setup in 10 steps, env vars, seed, demo URLs), architecture diagram (export from this doc), `/docs/youcam-usage.md` listing every endpoint used and why
- [ ] Hosted demo store URL + a judge guide page `/judge` with the three scripted prompts, a sample photo they may use, and a note on live vs cached renders
- [ ] Text description (under 400 words) in Devpost: problem, what we built, YouCam APIs used, what is novel, how to test, team
- [ ] 6–8 screenshots: launcher on store, onboarding photo step, Style DNA grid, outfit cards in chat, try-on viewer with before/after, swap drawer with scores, Crew view, cart drawer. Minimum 1280 px wide, no brand logos from third parties
- [ ] Demo video 2:30–2:55 on YouTube, public, no copyrighted music (use YouTube Audio Library or a royalty-free track), on-screen captions for every API call
- [ ] Share the project publicly (LinkedIn/X) for the Share & Earn bonus units
- [ ] Student submission: confirm the student account holder meets the Rising Star eligibility; use the alternate name, theme and store

21.2 Devpost description (draft)

Elara Commerce is a plug-and-play AI stylist widget for fashion e-commerce. A shopper uploads one full-body photo, tells the agent the occasion and budget in plain language, and gets complete outfits from that store's catalogue rendered on their own body with the YouCam AI Clothes Virtual Try-On API. Every piece can be swapped individually with a compatibility score; friends can join a Style Crew from a link, see the look on themselves and vote per item; the whole look goes to cart in one tap. The taste graph behind it (Style DNA) learns from every choice and travels with the shopper across stores. We used `cloth-v3` in three modes (single garment, layered full look, slot-level swap on an existing render), plus [bag/hat and background APIs if shipped]. Built in 36 hours on React, Node, Supabase and the YouCam API; demo store has 500 SKUs.

21.3 Video script (2:45)

Time	Shot	Voice-over	On-screen caption
0:00–0:12	Store homepage, cursor hovers products; cut to a phone screenshot of a WhatsApp chat "does this look ok on me??"	"Online fashion still asks you to imagine. We built a stylist that shows you instead, on any store, in one script tag."	Elara Commerce · YouCam eCommerce VTO Hackathon

Time	Shot	Voice-over	On-screen caption
0:12–0:30	Click "Style me"; onboarding: Me, Beach trip, ₹4,000; upload photo	"Three questions and one photo."	Onboarding: 2 screens, under 45 s
0:30–0:50	Style DNA tiles rendering on the uploaded photo, pick two	"Before you've bought anything, four looks on you. Your picks teach the agent your taste."	YouCam AI Clothes VTO · `garment_category: full_body`
0:50–1:20	Chat: agent returns three outfits with renders streaming in; open "Why this works"	"Now the agent styles complete outfits from this store's catalogue, within budget, and renders each one on you."	Multi-agent: intent → retrieve → compose → score → render
1:20–1:45	Tap swap on trousers; drawer with scores; pick one; only the bottom re-renders	"Don't like the trousers? Swap just that piece. The compatibility score explains why each option works, and only that part re-renders."	`garment_category: lower_body` on the existing render
1:45–2:10	Share with Crew → WhatsApp share image → friend opens link on phone, uploads photo, sees it on herself, comments "trousers yes"	"Shopping is social. Your crew joins from a link, sees it on themselves, and votes piece by piece."	Style Crew · per-item threads · realtime
2:10–2:20	Group photo on a beach background	"And one photo of the two of you, already there."	Group photo · YouCam Background Change (if shipped)
2:20–2:32	Gifting cut: "farewell outfit for my brother" → men's look, no gender question	"Buying for someone else? It infers who, and asks only what it must."	Recipient inference

Time	Shot	Voice-over	On-screen caption
2:32–2:40	Add to cart → cart drawer; Elara app constellation updates	"One tap to cart. Every choice feeds a Style DNA that follows you to the next store."	Taste graph
2:40–2:45	Embed snippet + feed JSON on screen; repo URL	"One script tag, any catalogue. Elara Commerce."	github.com/… · built with YouCam API

Recording notes: capture at 1920×1080 at 60 fps, hide the cursor trail, pre-warm the cache so renders land in 5–8 s on camera (the caption says "live render, cached" honestly), record the phone segment with a real phone on a tripod, and cut the final version to under 3:00 with 2 s of slack.

22. Risks and mitigations

The two risks most likely to sink the entry are running out of YouCam units before the video is recorded, and the layered full-look render producing poor results on separates. Both have a mitigation that must be in place by Sunday night, not Monday.

Risk	Likelihood	Impact	Mitigation	Owner
YouCam units exhausted before recording	Medium	Demo breaks	Read unit cost Sunday; cache everything; pre-render demo looks first; demo-mode fallback; second account's units reserved for recording and judging	Rishi
Layered render (bottom then top) looks wrong or `upper_body` re-render alters the trousers	Medium	Core claim weakens	Test on Sunday with 5 photos; if poor, render separates as a composed `full_body` reference (top + bottom stitched into one garment image) and keep swap as single-slot calls; show the better path in the video	Rishi

Risk	Likelihood	Impact	Mitigation	Owner
Catalogue arrives late or images are not VTO-friendly	Medium	Blocks ingestion and pre-renders	10-row sample by 2 PM Sunday; prefer flat-lay/ghost images; `vto_ready` flag so poor images are never chosen for rendering	Saksham
Two submissions judged as duplicates	Low–Medium	Disqualification or one entry ignored	Different names, themes, stores, descriptions and repos; different emphasis (general: merchant platform; student: social shopping); confirm rules allow a team member on two entries, else submit one	Both
Trademark or copyrighted material in video	Medium	Rule violation	Fictional store name; crop brand text from images; royalty-free music; no Zara/H&M/Myntra logos on screen	Saksham
Agent latency over 10 s before first text	Low	Feels broken	Stream status lines immediately; smaller model for intent; cap candidate pool	Rishi
Realtime for Crew flaky on free tier	Low	Crew demo stalls	5 s polling fallback; record crew segment with both browsers on one machine	Rishi
Judge uploads a photo that fails YouCam input rules	Medium	Bad first impression	Client-side checks with clear guidance; sample models always available	Saksham

Risk	Likelihood	Impact	Mitigation	Owner
Deadline confusion (Devpost shows 10:15 PM IST on 2 Nov; planning notes said 8 PM PT)	Low	Missed submission	Treat 10:15 PM IST as the deadline; submit by 9 PM IST	Both
Scope creep (Studio, wardrobe, accessories) eats P0 time	High	Nothing is finished	Tiers in Section 6; freeze at 3 PM Monday; P2 only as video mocks	Both

23. Open decisions needed from the team

Ten decisions were left open in the planning call or are new in this PRD. The default is what the build follows if no one objects; the first four need an answer on Sunday.

#	Decision	Default in this PRD	Needed by
1	Product name for the general-track entry, and a different name for the student entry	Elara Commerce; student entry name TBD	Sun 6 PM
2	YouCam unit cost per Clothes VTO call (read from console) and the resulting live-render allowance	Plan assumes ⩽7 units/call	Sun 6 PM
3	`cloth-v3` vs `cloth-v4` on the hackathon key	v3, documented and stable	Sun 6 PM
4	Demo store brand and catalogue source (Myntra vs Zara vs H&M images)	Fictional resort-wear brand, Myntra listings, brand text cropped	Sun 2 PM
5	Does the team member who is a student submit alone on the student track, and is one shared repo allowed across both entries? Check the Devpost rules page	Two repos, two accounts, different UI	Sun 10 PM

#	Decision	Default in this PRD	Needed by
6	Full-look rendering: layered calls (bottom then top) vs one composed reference image	Layered; switch if Sunday tests look poor	Sun 10 PM
7	Should footwear be rendered when the look includes shoes (`change_shoes: true`), given extra realism risk	Render shoes only via a separate `shoes` call if the store sells footwear; otherwise keep the user's shoes	Mon 10 AM
8	Style DNA tiles: live render for every uploaded photo, or pre-rendered for sample models only	Live for uploads, pre-rendered for sample models	After #2
9	Group photo in scope (needs background-removal + background-change calls)	P2; video shows it if built by freeze	Mon 3 PM
10	LLM provider for agents (Claude via API vs OpenAI vs the current Elara stack)	Whatever Elara already runs; JSON-schema tool calling required	Sun 2 PM

Questions the planning notes left unanswered that this PRD has decided (change if wrong): how many Style DNA looks to show (four), what feedback controls appear (heart/cross, skip), how a dislike affects results (halves the vibe weight; two dislikes → never list), how the compatibility score is computed (Section 11.4), and which accessory APIs count toward "three capabilities" (bag and hat, P2).

24. Appendix

A. Intent parser output schema

```
{
  "for": "self" | "other",
  "recipient": { "relation": "brother", "gender": "male" | "female" | null,
"age_range": "18-24" | null },
  "occasion": "beach trip",
  "formality_target": 1,
  "vibes": ["minimal", "breezy"],
  "budget_max": 4000,
  "currency": "INR",
  "must_include": ["linen shirt"],
  "exclude": ["black"],
  "modify_look_id": null,
  "missing_required": ["gender"]
}
```

B. Composer system prompt (abridged)

You are a personal stylist for one store. You receive a brief, a taste vector summary and a candidate pool grouped by slot. Return 2–3 complete outfits as JSON. Rules: every outfit has a top and bottom, or a one-piece; add a layer only if the occasion or weather calls for it; add footwear, bag or accessory only if candidates exist in that slot; total price must be at or under budget_max; at most one statement piece per outfit; colours must satisfy the harmony rules supplied; never invent products, use only candidate ids; give each outfit a two-word name and one sentence on why it works for this person and occasion, in second person, no exclamation marks.

C. Clarifier decision table

Missing	Store is single-gender	Ask?	Chips
occasion	any	Yes	Beach · Wedding · Office · Date · Everyday
gender (self)	yes	No	—
gender (self)	no, not inferable	Yes	Women · Men · Show both
gender (recipient)	no, relation gives it	No	—
budget	any	Only if phrasing implies a cap without a number	Under ₹2k · ₹2–4k · ₹4–8k · No limit
anything else	any	No	—

D. YouCam wrapper interface

```
export type GarmentCategory = 'full_body' | 'upper_body' | 'lower_body' |
'shoes' | 'auto';
export interface TryOnRequest { srcFileId: string; refUrl: string; category:
GarmentCategory; changeShoes?: boolean; sessionId: string; }
export interface TryOnResult { imageUrl: string; taskId: string; cached:
boolean; unitsUsed: number; latencyMs: number; }
export async function tryOn(req: TryOnRequest): Promise<TryOnResult>; //
cache → budget check → POST task → poll → store image
export async function uploadUserPhoto(buffer: Buffer, mime: string):
Promise<{ fileId: string }>;
export function remainingUnits(): Promise<number>;
```

E. Compatibility reason templates

- Colour lead: "Same muted palette as the rest of the look" / "Neutral, so it goes with anything here"
- Formality lead: "Matches the relaxed beach level" / "A notch dressier, which lifts the whole look"
- Silhouette lead: "Straighter cut balances the loose top"
- Trade-off suffix: "…though it pushes the total ₹300 over budget" / "…but a bolder colour than you usually pick"

F. Glossary

Term	Meaning
Look	A complete outfit: named set of products, one per slot, with a total and a render
Slot	top, bottom, one-piece, layer, footwear, bag, accessory
Render	One image produced by the YouCam API for a (photo, garment, category) triple
Swap	Replacing one slot of a look and re-rendering only that slot
Style DNA	The shopper's taste profile: vibe and attribute weights built from choices
Taste graph	The node/edge store behind Style DNA
Crew	A shared group around one or more looks, joined by link

Term	Meaning
Demo mode	Serving cached YouCam renders when the key is exhausted or slow

G. Source material

Planning-call notes (Granola, 31 Oct), the original idea note, TALA integration screenshots (chat, studio, before/after), Elara app screenshots (Style DNA constellation, wardrobe, item detail), the Devpost hackathon page, and the YouCam docs linked in Section 15.